Graphic Processing Project

Student: Simu Darius
Group: 30435

January 8, 2026

Table of Contents

1 Contents **3**

2 Subject Specification **3**
2.1 Requirements . . . 3
2.2 Objectives . . . 5

3 Scenario **5**
3.1 Scene and Objects Description . . . 5
3.1.1 Main Components . . . 5
3.1.2 Object Properties . . . 6
3.2 Functionalities . . . 7
3.2.1 Shader Implementation Details . . . 9

4 Implementation Details **10**
4.1 Functions and Special Algorithms . . . 10
4.1.1 Collision Detection Algorithm . . . 10
4.1.2 Collision Detection Implementation: . . . 11
4.1.3 Shadow Mapping Algorithm . . . 13
4.1.4 Camera Path Interpolation with State Machine . . . 16
4.2 Graphics Model . . . 17
4.2.1 Rendering Pipeline . . . 17
4.2.2 Coordinate Systems . . . 18
4.2.3 Transformations . . . 18
4.3 Data Structures . . . 19
4.4 Class Hierarchy . . . 20
4.4.1 Main Classes . . . 20

5 Graphical User Interface Presentation / User Manual **22**
5.1 Interface Overview . . . 22
5.2 User Manual . . . 23
5.2.1 Getting Started . . . 23
5.2.2 Basic Operations . . . 24
5.2.3 Keyboard Controls . . . 24
5.2.4 Mouse Controls . . . 24

6 Conclusions and Further Developments **24**
6.1 Conclusions . . . 24
6.2 Further Developments . . . 25

7 References **25**

1 Contents

This document describes the design, implementation, and functionality of an OpenGL application. The project aims to render a photorealistic 3D scene with lighting, shadowing, collision detection, and an automated presentation mode to showcase all the features implemented in the project.

The document is organized as follows:

- Section 2 presents the subject specification
- Section 3 describes the scenario, scenes, and objects
- Section 4 details the implementation approach
- Section 5 presents the graphical user interface
- Section 6 discusses conclusions and future work

2 Subject Specification

The project implements the render of a photorealistic 3D scene using modern OpenGL techniques, simulating an atmospheric dungeon environment with advanced lighting, shadows, and interactive camera controls. The application demonstrates real-time rendering capabilities while maintaining visual fidelity through multiple shadow mapping techniques, collision detection, and dynamic lighting systems.

The primary objective is to create an immersive 3D environment that showcases technical proficiency in computer graphics while providing both manual exploration and automated presentation modes. The scene features a medieval dungeon setting with architectural elements, dynamic lighting from multiple sources, and atmospheric effects enhanced by shadow computations.

2.1 Requirements

The project fulfills the following technical requirements:

- **Scene Visualization:** Complete implementation of camera controls including:
  - WASD keyboard navigation with collision detection
  - Mouse-based camera rotation with pitch and yaw constraints
  - Automated presentation mode (B key) following a predefined path with 16 path positions, of witch 5 have look-at functions
  - Smooth camera interpolation and rotation during automated sequences
- **Light Sources:** Implementation of multiple distinct lighting systems:
  - Dynamic spotlight (flashlight) attached to camera position and direction
  - Static point light (torch) with omnidirectional illumination at position (-40.14, 7.05, 81.3)

  - Toggle-able flashlight (F key) for interactive lighting control

- **Rendering Modes:** Support for multiple visualization modes:
  - Solid rendering with Phong shading and material properties
  - Wireframe collision mesh overlay (C key toggle) for debugging spatial structures
  - Smooth surface rendering using normal interpolation

- **Textures and Materials:** Advanced material system featuring:
  - High-resolution texture mapping (diffuse, specular, normal maps)
  - Material properties including ambient, diffuse, and specular components
  - Proper UV coordinate mapping on architectural models
  - LOD-appropriate texture resolution for performance optimization

- **Shadow Computation:** Dual shadow mapping system:
  - 2D perspective shadow mapping for directional spotlight (8192×8192 resolution)
  - Cubemap omnidirectional shadow mapping for point light source
  - PCF (Percentage Closer Filtering) for soft shadow edges
  - Dynamic shadow updates for moving light sources

- **Animation:** Multiple animation systems:
  - Smooth camera path interpolation during presentation mode
  - Ghost character movement with waypoint navigation
  - Camera look-at sequences with timed pauses at points of interest
  - Synchronized timing for narrative presentation (ghost appearances, turns)

- **Advanced Features:** Enhanced realism through:
  - Spatial hash grid collision detection for efficient player-environment interaction
  - Detailed architectural modeling with over 10000+ triangles in collision mesh
  - Photorealistic lighting combining spotlight and point light with shadow interactions
  - Atmospheric presentation mode with cinematic camera movements and scripted events
  - State machine-based animation system for complex camera behavior sequences
  - High-resolution shadow maps (8192×8192) for sharp, artifact-free shadows

2.2 Objectives

The project aims to achieve the following technical and artistic objectives:

1. **Technical Implementation:** Apply concepts from the modern OpenGL rendering pipeline, including shadow mapping techniques (using a 2D map for a spotlight and a cubemap for a point light), basic spatial structures for collision detection, and standard rendering optimizations

2. **Visual Fidelity:** Create a photorealistic dungeon environment that effectively uses lighting, shadows, and materials to establish atmosphere and mood, with attention to texture quality and lighting interaction

3. **Interactive Experience:** Provide intuitive manual controls for scene exploration while offering an automated presentation mode that guides viewers through sequences highlighting the technical and artistic features

4. **Performance Optimization:** Maintain real-time frame rates through efficient algorithms (spatial hash grid for O(1) collision queries), pre-computed static shadows for the torch light, and optimized rendering passes

5. **Extensible Architecture:** Design modular code structure with separate shader programs for different rendering passes, reusable model loading systems, and flexible camera control mechanisms that facilitate future enhancements

3 Scenario

The application presents an atmospheric medieval dungeon environment designed to demonstrate advanced real-time rendering techniques. The user can freely explore the dungeon corridors using first-person controls, or activate an automated cinematic presentation that guides them through a scripted narrative sequence featuring architectural landmarks, lighting demonstrations, and a supernatural encounter.

The scene is set in a multi-level stone dungeon with winding corridors, staircases, prison cells, and a torch-lit chamber. The environment showcases two distinct lighting systems working in tandem: a player-controlled flashlight that illuminates the immediate surroundings, and a static torch mounted on the wall that casts a warm light throughout its chamber. Both light sources cast realistic shadows using different shadow mapping techniques appropriate to their characteristics.

3.1 Scene and Objects Description

The virtual environment consists of architectural elements constructed from a detailed 3D tileset and an animated ghost character that appears during the presentation sequence. All the objects and models used in the virtual scene were designed in Blender.

3.1.1 Main Components

- **Dungeon Architecture (TileSet)**: The primary geometric structure consisting of stone walls, floors, ceilings, staircases, and other objects that aim to strengthen the immersion of the scene. Loaded from **models/TileSet/TileSetGP.obj**, this

model forms the static collision geometry and visual foundation of the scene. The architecture features:

  - Ground floor corridors with prison cells
  - 90-degree corner turns
  - A staircase connecting two vertical levels
  - Upper-level walkway overlooking the main chamber
  - Total collision mesh: 10000+ triangles organized in a spatial hash grid

- **Ghost Character**: An animated supernatural entity loaded from **models/Ghost/Ghost.obj** that appears during the automated presentation. The ghost demonstrates:

  - Waypoint-based movement system with smooth interpolation
  - Three distinct movement sequences synchronized with camera positions
  - Initial position: (-36.5, 0.5, 83.5) at the end of the main chamber
  - Visibility toggling for dramatic appearances/disappearances
  - Rotation capability for directional facing (initially -90° Y-axis)

- **Lighting System**: Dual-source dynamic lighting implementation:

  - *Flashlight (Spotlight)*: Player-attached directional light with 60° field of view, following camera position and orientation. Features smooth falloff (cutoff at 17°, outer cutoff at 15°) and toggle functionality (F key)
  - *Fallen Torch (Point Light)*: Static omnidirectional light at position (-40.14, 7.05, 81.3) emitting warm orange light (RGB: 1.0, 0.5, 0.0). Illuminates the upper chamber with quadratic attenuation

- **Shadow Mapping Systems**:

  - *2D Perspective Shadow Map*: 8192×8192 resolution depth texture for flashlight shadows using perspective projection. Implements PCF sampling and bias correction for artifact reduction
  - *Cubemap Shadow Map*: 8192×8192 per-face omnidirectional depth cubemap for torch shadows. Pre-computed during initialization for performance optimization

3.1.2 Object Properties

- **Camera Properties:**

  - Initial Position: (-36.5, 2.0, 114.5) facing forward along Z-axis
  - Movement Speed: 0.05 units per frame
  - Mouse Sensitivity: 0.1 degrees per pixel
  - Pitch Range: Constrained between -89° and +89° to prevent gimbal lock
  - Collision Radius: 0.3 units (spherical approximation)
  - Projection: 60° vertical FOV, near plane 0.1, far plane 50.0

- **Ghost Properties:**
  - Movement Speed: 0.07 units per frame
  - Target Waypoints:
    1. Target 1: (-59.5, 0.5, 83.5) - First appearance in the main chamber
    2. Target 2: (-59.5, 0.5, 88.5) - Cell area vanishing point
    3. Target 3: (-42.5, 6.5, 62.0) - Final approach toward camera on the upper walkway
  - Rendering: Affected by the flashlight and casts shadows
- **Lighting Properties:**

  Flashlight (Spotlight):
  - Color: White (1.0, 1.0, 1.0) when active, Black (0.0, 0.0, 0.0) when toggled off
  - Position Offset: 0.5 units forward, 0.5 units down from camera
  - Direction: Aligned with camera front vector (minimal downward tilt: 0.00005)
  - Cone Angle: Inner 15°, Outer 17° with smooth intensity falloff
  - Attenuation: Quadratic (constant: 1.0, linear: 0.022, quadratic: 0.0019)
  - Shadow Map Resolution: 8192×8192, updated every frame

  Torch (Point Light):
  - Color: Warm Orange (1.0, 0.5, 0.0)
  - Position: Fixed at (-40.14, 7.05, 81.3)
  - Attenuation: Quadratic (constant: 1.0, linear: 0.045, quadratic: 0.0075)
  - Shadow Far Plane: 50.0 units
  - Shadow Map: Pre-computed 6-face cubemap (8192×8192 per face)
- **Atmospheric Effects:**
  - Fog Density: 0.1
  - Fog Color: Near-black (0.001, 0.001, 0.001) for dark and claustrophobic atmosphere
  - Ambient Light: Minimal (0.007, 0.007, 0.007) to maintain dungeon mood

3.2 Functionalities

The application provides comprehensive interaction capabilities through keyboard and mouse controls, along with an automated demonstration mode.

1. **Manual Camera Navigation**: Free first-person exploration with full 6-DOF movement:
   - WASD keys for forward/backward/strafe movement
   - Mouse movement for pitch/yaw camera rotation

- Collision detection prevents clipping through walls and geometry
- Spatial hash grid optimization enables real-time collision queries
- Movement speed: 0.05 units/frame with collision response

2. **Automated Presentation Mode (B Key)**: Cinematic camera path following a predefined 16-point spline with scripted events:

   - *Point 0-1*: Initial corridor walk into the main chamber with ghost movement starting at distance ¡ 2.5 units
   - *Point 1*: A quick pause to look at the corridor the ghost fled into
   - *Point 2-3*: Walking next to a wall with different weapon stands that are being lit by the light of the torch on the upper walkway
   - *Point 3*: Short pause and a diagonal look sequence to show a statue of a knight then continuing forward
   - *Point 4*: Walking into the left corridor with ghost movement to target 2 at midpoint
   - *Point 5*: Look-left to observe prison cell interior, pause, and return
   - *Point 6-7*: Corner navigation with smooth rotation
   - *Point 8-9*: First staircase ascent (3 unit vertical climb)
   - *Point 10-11*: Second corner navigation
   - *Point 12-13*: Second staircase ascent (3 unit vertical climb)
   - *Point 14*: Diagonal look to observe the hanging skeleton
   - *Point 15*: Final position with 180 degree turn, ghost spawn at (-42.5, 6.5, 72.0) and movement toward camera upon turn completion

   State machine controls each look sequence with phases: LOOK → PAUSE (1 second) → LOOK_BACK → DONE. Camera interpolation speed: 0.02 units/frame (movement), 0.01 radians/frame (rotation).

3. **Flashlight Toggle (F Key)**: Dynamic control of the player's spotlight:

   - Instant on/off switching without performance impact
   - Light color switches between white (1,1,1) and black (0,0,0)
   - Shadow map continues updating regardless of toggle state
   - Console feedback: "Flashlight: ON" or "Flashlight: OFF"

4. **Collision Wireframe Visualization (C Key)**: Debug mode displaying the spatial collision geometry:

   - Green wireframe overlay (RGB: 0, 1, 0) rendered on top of scene
   - Shows all triangles in the collision mesh using GL_LINES mode
   - Visualizes spatial hash grid coverage and triangle distribution
   - Useful for debugging collision detection and understanding geometry complexity

- No performance impact when disabled

5. **Real-time Shadow Computation**: Automatic shadow generation from both light sources:
   - Flashlight shadows: Updated every frame in render pass 1 (2D depth map)
   - Torch shadows: Pre-computed once during initialization (cubemap depth)
   - Shadow bias adjustment: Adaptive bias based on surface angle to prevent acne
   - PCF sampling: Soft shadow edges through percentage closer filtering
6. **Fog Rendering**: Distance-based atmospheric depth cueing:
   - Exponential squared falloff: `exp(-pow(distance * 0.1, 2.0))`
   - Calculated in eye space for consistent appearance regardless of orientation
   - Blends scene color toward near-black fog color (0.001, 0.001, 0.001)
   - Enhances depth perception and creates cave-like atmosphere
   - Implemented in fragment shader for better accuracy
7. **Multi-pass Rendering Pipeline**: Optimized three-pass rendering architecture:
   - *Pass 1 (Flashlight Shadows)*: Renders scene to 8192×8192 2D depth texture from light's perspective using `depthMap2D.vert/frag` shaders. Viewport: full shadow resolution. Face culling: front faces to reduce shadow acne.
   - *Pass 2 (Torch Shadows)*: One-time pre-computation rendering scene 6 times to cubemap faces using `depthMapCube.vert/frag` shaders. Each face uses 90° FOV perspective with appropriate `lookAt` matrix. Result cached for subsequent frames.
   - *Pass 3 (Final Render)*: Main scene rendering to screen framebuffer at window resolution using `basic.vert/frag` shaders. Samples both shadow maps, combines lighting from both sources, applies fog, outputs to screen.

3.2.1 Shader Implementation Details

Fragment Shader Lighting Model (`basic.frag`) The fragment shader implements the lighting model combining two distinct light sources with shadow mapping and atmospheric fog:

```
// Flashlight (Spotlight) with Cone Attenuation:
vec3 L_flash = normalize(lightPos - fWorldPos);
float theta = dot(normalize(fWorldPos - lightPos), normalize(lightDir));
float epsilon = cos(radians(15.0)) - cos(radians(17.0));
float intensity = clamp((theta - cos(radians(17.0))) / epsilon, 0.0, 1.0);
float attenF = 1.0 / (1.0 + 0.022*dist + 0.0019*dist²);

// Torch (Point Light) with 360° Cubemap Shadow:
float shadowT = compute360Shadow(fWorldPos);
vec3 torchResult = (diffT * (1.0 - shadowT)) * torchColor * attenT;

// Combined Result with Minimal Ambient:
vec3 finalRGB = (flashlightResult + torchResult + 0.007) * objectColor;
```

Key Shader Features

- **Dual Shadow Sampling**: Separate functions `computeFlashlightShadow()` (2D) and `compute360Shadow()` (cubemap) with different bias strategies
- **Adaptive Shadow Bias**: Flashlight uses angle-based bias: `max(0.00005*(1-dot(N,L)), 0.00001)` to prevent both acne and peter-panning
- **Border Clamping**: Shadow map coordinates checked against [0,1] range to prevent out-of-bounds sampling
- **Distance-based Attenuation**: Different coefficients for flashlight (0.022 linear, 0.0019 quadratic) vs torch (0.045 linear, 0.0075 quadratic)

Geometry Processing (`basic.vert`)

- Transforms vertex positions to world space for lighting calculations
- Computes normal matrix as `transpose(inverse(model))` for non-uniform scaling support
- Generates light-space coordinates: `lightSpaceTrMatrix * vec4(fWorldPos, 1.0)`
- Passes through texture coordinates unchanged

Shadow Map Generation Shaders

- `depthMap2D.vert`: Simple transformation to light space, no color output
- `depthMapCube.vert/frag`: Outputs world position, calculates normalized distance to light source in fragment shader for correct cubemap depth storage

4 Implementation Details

The implementation leverages modern OpenGL Core Profile features combined with efficient data structures to achieve real-time performance. The architecture consists of multiple modular components: rendering pipeline, collision detection system, camera control, and a model loading system.

4.1 Functions and Special Algorithms

Three algorithms form the foundation of the application: collision detection, shadow mapping, and automated camera path interpolation.

4.1.1 Collision Detection Algorithm

Requirement: Prevent the camera from penetrating solid geometry in a complex mesh with 10000+ triangles while maintaining good performance.

To achieve this a **Spatial Hash Grid** is used meaning we partition the 3D space into uniform grid cells and store triangles in cells they overlap.

Advantages:

- O(1) average-case lookup using hash function
- Simple implementation: single hash map data structure
- Efficient for localized queries (camera collision radius = 0.3 units)
- No tree balancing or complex maintenance
- Minimal memory overhead

Disadvantages:

- Hash collisions possible (mitigated with large key space)
- Memory proportional to triangle count and overlap
- Requires choosing appropriate cell size

4.1.2 Collision Detection Implementation:

```
struct SpatialGrid {
    float cellSize;
    std::unordered_map<int64_t, std::vector<Triangle>> grid;

    int64_t hashPos(int x, int y, int z) {
        return ((int64_t)x << 32) | ((int64_t)y << 16) | (int64_t
            )z;
    }

    void insert(const Triangle& tri) {
        glm::vec3 minPos = glm::min(glm::min(tri.v0, tri.v1), tri
            .v2);
        glm::vec3 maxPos = glm::max(glm::max(tri.v0, tri.v1), tri
            .v2);

        int minX = (int)floor(minPos.x / cellSize);
        int minY = (int)floor(minPos.y / cellSize);
        int minZ = (int)floor(minPos.z / cellSize);
        int maxX = (int)floor(maxPos.x / cellSize);
        int maxY = (int)floor(maxPos.y / cellSize);
        int maxZ = (int)floor(maxPos.z / cellSize);

        for (int x = minX; x <= maxX; x++) {
            for (int y = minY; y <= maxY; y++) {
                for (int z = minZ; z <= maxZ; z++) {
                    grid[hashPos(x, y, z)].push_back(tri);
                }
            }
        }
    }

    std::vector<Triangle> queryNearby(glm::vec3 pos, float radius
        ) {
```

```
        std::vector<Triangle> nearby;

        int minX = (int)floor((pos.x - radius) / cellSize);
        int minY = (int)floor((pos.y - radius) / cellSize);
        int minZ = (int)floor((pos.z - radius) / cellSize);
        int maxX = (int)floor((pos.x + radius) / cellSize);
        int maxY = (int)floor((pos.y + radius) / cellSize);
        int maxZ = (int)floor((pos.z + radius) / cellSize);

        for (int x = minX; x <= maxX; x++) {
            for (int y = minY; y <= maxY; y++) {
                for (int z = minZ; z <= maxZ; z++) {
                    auto it = grid.find(hashPos(x, y, z));
                    if (it != grid.end()) {
                        nearby.insert(nearby.end(), it->second.begin(), it->second.end());
                    }
                }
            }
        }

        return nearby;
    }
};
```

Listing 1: Spatial Grid Structure

```
void extractCollisionMesh(gps::Model3D& model3d) {
    std::cout << "Extracting collision mesh..." << std::endl;

    spatialGrid.cellSize = 1.0f;

    int totalTriangles = 0;

    for (const auto& mesh : model3d.meshes) {
        for (size_t i = 0; i < mesh.indices.size(); i += 3) {
            if (i + 2 < mesh.indices.size()) {
                Triangle tri;
                tri.v0 = mesh.vertices[mesh.indices[i]].Position;
                tri.v1 = mesh.vertices[mesh.indices[i + 1]].Position;
                tri.v2 = mesh.vertices[mesh.indices[i + 2]].Position;

                spatialGrid.insert(tri);
                allCollisionTriangles.push_back(tri);
                totalTriangles++;
            }
        }
    }

    std::cout << "Extracted " << totalTriangles << " triangles
```

```
        into spatial grid" << std::endl;
    std::cout << "Grid cells used: " << spatialGrid.grid.size()
        << std::endl;
}
```

Listing 2: Collision Mesh Extraction

```
bool checkMeshCollision(glm::vec3 position) {
    glm::mat4 inverseModel = glm::inverse(model);
    glm::vec3 localPos = glm::vec3(inverseModel * glm::vec4(
        position, 1.0f));

    std::vector<Triangle> nearbyTris = spatialGrid.queryNearby(
        localPos, cameraRadius);

    for (const auto& tri : nearbyTris) {
        float dist = pointToTriangleDistance(localPos, tri);
        if (dist < cameraRadius) {
            return true;
        }
    }

    return false;
}
```

Listing 3: Collision Detection

4.1.3 Shadow Mapping Algorithm

Requirement: Compute realistic shadows for two light sources with distinct characteristics:

- a *dynamic spotlight* (flashlight) attached to the camera
- a *static point light* (torch) emitting light in all directions

To satisfy these requirements efficiently, a **Dual Shadow Mapping System** is employed, selecting the most suitable shadow technique for each light type.

Flashlight: Perspective Shadow Mapping The flashlight is modeled as a spotlight with a limited cone angle and dynamic position and orientation.

- **Shadow Technique:** 2D perspective shadow mapping
- **Depth Map Resolution:** 8192×8192
- **Justification:** The spotlight's pyramidal frustum naturally matches a perspective projection.
- **Update Frequency:** Every frame, as the light follows camera movement

Light Space Transformation. The light-space matrix is constructed using a perspective projection aligned with the spotlight cone and a view matrix oriented along the light direction. Special handling is included to avoid singularities when the light direction aligns with the world up vector.

```
glm::mat4 computeLightSpaceTrMatrix() {
    const GLfloat near_plane = 0.1f, far_plane = 50.0f;
    // Perspective projection for flashlight cone
    glm::mat4 lightProjection = glm::perspective(glm::radians
       (60.0f), 1.0f, near_plane, far_plane);
    // Create view matrix
    glm::vec3 lightTarget = lightPos + lightDir;
    // Choose up vector
    glm::vec3 up = glm::vec3(0.0f, 1.0f, 0.0f);
    // Use different up vector if too parallel
    if (glm::abs(glm::dot(glm::normalize(lightDir), up)) > 0.95f)
        {
        up = glm::vec3(1.0f, 0.0f, 0.0f);
    }
    glm::mat4 lightView = glm::lookAt(lightPos, lightTarget, up);

    return lightProjection * lightView;
}
```

Listing 4: Flashlight Light-Space Matrix Computation

Torch: Omnidirectional Shadow Mapping The torch is modeled as a point light emitting uniformly in all directions and remaining static in the scene.

- **Shadow Technique:** Omnidirectional shadow mapping using a depth cubemap
- **Resolution:** 8192×8192 per cubemap face
- **Justification:** A cubemap naturally captures the full 360° light emission of a point light.
- **Update Frequency:** Precomputed once during initialization
- **Optimization:** Avoids six shadow rendering passes per frame

Cubemap Shadow Generation. A 90° perspective projection is used for each cubemap face. Six view matrices are generated, corresponding to the positive and negative directions of the three coordinate axes.

```
float near_plane = 0.1f;
float far_plane = 150.0f;
glm::mat4 shadowProj = glm::perspective(glm::radians(90.0f), 1.0f
   , near_plane, far_plane);

std::vector<glm::mat4> shadowTransforms;
shadowTransforms.push_back(shadowProj * glm::lookAt(torchPos,
   torchPos + glm::vec3(1, 0, 0), glm::vec3(0, -1, 0)));
```

```
shadowTransforms.push_back(shadowProj * glm::lookAt(torchPos, torchPos + glm::vec3(-1, 0, 0), glm::vec3(0, -1, 0)));
shadowTransforms.push_back(shadowProj * glm::lookAt(torchPos, torchPos + glm::vec3(0, 1, 0), glm::vec3(0, 0, 1)));
shadowTransforms.push_back(shadowProj * glm::lookAt(torchPos, torchPos + glm::vec3(0, -1, 0), glm::vec3(0, 0, -1)));
shadowTransforms.push_back(shadowProj * glm::lookAt(torchPos, torchPos + glm::vec3(0, 0, 1), glm::vec3(0, -1, 0)));
shadowTransforms.push_back(shadowProj * glm::lookAt(torchPos, torchPos + glm::vec3(0, 0, -1), glm::vec3(0, -1, 0)));

glViewport(0, 0, SHADOW_WIDTH, SHADOW_HEIGHT);
glBindFramebuffer(GL_FRAMEBUFFER, torchShadowMapFBO);

depthCubeShader.useShaderProgram();
glUniform1f(
    glGetUniformLocation(depthCubeShader.shaderProgram, "far_plane"),
    far_plane
);
glUniform3fv(
    glGetUniformLocation(depthCubeShader.shaderProgram, "lightPos"),
    1, glm::value_ptr(torchPos)
);

for (unsigned int i = 0; i < 6; i++) {
    glFramebufferTexture2D(
        GL_FRAMEBUFFER,
        GL_DEPTH_ATTACHMENT,
        GL_TEXTURE_CUBE_MAP_POSITIVE_X + i,
        torchDepthMapTexture,
        0
    );
    glClear(GL_DEPTH_BUFFER_BIT);

    glUniformMatrix4fv(
        glGetUniformLocation(depthCubeShader.shaderProgram, "lightSpaceTrMatrix"),
        1, GL_FALSE, glm::value_ptr(shadowTransforms[i])
    );

    drawObjects(depthCubeShader, true);
}

glBindFramebuffer(GL_FRAMEBUFFER, 0);
```

Listing 5: Torch Cubemap Shadow Map Generation

4.1.4 Camera Path Interpolation with State Machine

Requirement: Create cinematic camera movements that smoothly follow a path while performing complex look-at sequences. In order to achieve the desired cinematic experience we use a **State Machine with Interpolation** system, each "look around" point using a finite state machine with phases.

```
enum P5State { P5_NONE, P5_LOOK, P5_PAUSE, P5_LOOK_BACK, P5_DONE };
P5State currentP5State = P5_NONE;

// In updatePresentation():
if (currentP5State != P5_NONE && currentP5State != P5_DONE) {
    if (currentP5State == P5_LOOK) {
        glm::vec3 leftDir = glm::normalize(glm::cross(glm::vec3(0, 1, 0), savedDirection));
        myCamera.cameraFrontDirection = glm::normalize(glm::mix(myCamera.cameraFrontDirection, leftDir, presentationRotationSpeed));
        if (glm::dot(myCamera.cameraFrontDirection, leftDir) > 0.989f) {
            currentP5State = P5_PAUSE;
            timer = glfwGetTime();
        }
    }
    else if (currentP5State == P5_PAUSE) {
        if (glfwGetTime() - timer > 1.0f) currentP5State = P5_LOOK_BACK;
    }
    else if (currentP5State == P5_LOOK_BACK) {
        myCamera.cameraFrontDirection = glm::normalize(glm::mix(myCamera.cameraFrontDirection, savedDirection, presentationRotationSpeed));
        if (glm::dot(myCamera.cameraFrontDirection, savedDirection) > 0.999f) {
            currentP5State = P5_DONE;
            currentPathIndex++;
        }
    }
    view = myCamera.getViewMatrix();
    myBasicShader.useShaderProgram();
    glUniformMatrix4fv(viewLoc, 1, GL_FALSE, glm::value_ptr(view));
    return;
}
```

Listing 6: State Machine Example (Point 5)

Key Features:

- **Smooth Rotation:** `glm::mix(current, target, speed)` provides ease-in/ease-out

- **Precision Control:** Threshold values for exact alignment
- **Non-blocking:** Each state returns after update, allowing ghost animations to run in parallel
- **Saved Direction:** Stores original forward vector before looking around for accurate return

4.2 Graphics Model

4.2.1 Rendering Pipeline

The application uses a three-pass deferred-style rendering architecture:

Pass 1: Flashlight Shadow Map Generation

1. Bind shadow framebuffer (8192×8192 depth texture)
2. Enable front-face culling to reduce shadow acne
3. Render scene from light's perspective using `depthMap2D` shader
4. Store depth values in 2D texture bound to unit 3

Pass 2: Torch Shadow Cubemap Generation (One-time)

1. For each of 6 cubemap faces (+X, -X, +Y, -Y, +Z, -Z):
   - Attach face to framebuffer depth attachment
   - Render scene with 90° FOV perspective
   - Store normalized distance (0-1 range) in fragment shader
2. Cubemap bound to texture unit 4

Pass 3: Final Scene Rendering

1. Bind default framebuffer (screen)
2. Enable back-face culling
3. Vertex Shader:
   - Transform vertices to clip space
   - Compute light-space coordinates for shadow sampling
   - Pass world position, normals, texture coordinates to fragment shader
4. Fragment Shader:
   - Sample both shadow maps (2D for flashlight, cubemap for torch)
   - Compute lighting for both sources independently
   - Combine results with additive blending
   - Apply exponential squared fog
   - Output final color

4.2.2 Coordinate Systems

Object Space → World Space:

```
// Vertex Shader
fWorldPos = vec3(model * vec4(vPosition, 1.0));
```

World Space → View Space:

```
fPosEye = view * model * vec4(vPosition, 1.0);
```

World Space → Light Space (Flashlight):

```
fragPosLightSpace = lightSpaceTrMatrix * vec4(fWorldPos, 1.0);
```

World Space → Clip Space:

```
gl_Position = projection * view * model * vec4(vPosition, 1.0);
```

Coordinate System Properties:

- **World Space:** Right-handed, Y-up, Z-forward
- **View Space:** Camera at origin, looking down -Z axis
- **Light Space:** Perspective projection for flashlight, 90° FOV for each cubemap face
- **Clip Space:** Normalized device coordinates $[-1,1]^3$ after perspective divide

4.2.3 Transformations

Model Matrix: Identity for static dungeon, transformation matrix for ghost:

```
glm::mat4 ghostMatrix = glm::translate(glm::mat4(1.0f), ghostPos)
    ;
ghostMatrix = glm::rotate(ghostMatrix, glm::radians(
    ghostRotationAngle), glm::vec3(0.0f, 1.0f, 0.0f));
```

View Matrix: Updated by camera class using `glm::lookAt`:

```
glm::mat4 Camera::getViewMatrix() {
    return glm::lookAt(cameraPosition, cameraPosition +
        cameraFrontDirection, cameraUpDirection);
}
```

Projection Matrix: Perspective with 60° vertical FOV:

```
const GLfloat near_plane = 0.1f, far_plane = 50.0f;
glm::mat4 lightProjection = glm::perspective(glm::radians(60.0f),
    1.0f, near_plane, far_plane);
```

Normal Matrix: Corrects normals for non-uniform scaling:

```
glm::mat3 normalMatrix = glm::mat3(glm::inverseTranspose(view *
    model));
```

4.3 Data Structures

Core Geometric Structures:

```
struct Vertex {
    glm::vec3 Position;
    glm::vec3 Normal;
    glm::vec2 TexCoords;
};

struct Triangle {
    glm::vec3 v0, v1, v2;
};

struct Texture {
    GLuint id;
    std::string type;
    std::string path;
};
```

Listing 7: Vertex and Triangle Structures

Mesh Buffers:

```
struct Buffers {
    GLuint VAO;
    GLuint VBO;
    GLuint EBO;
};
```

Listing 8: OpenGL Buffer Objects

Camera Path Data:

```
std::vector<glm::vec3> presentationPath = {
    glm::vec3(-36.5f, 2.0f, 114.5f),  // Point 0: Start
    glm::vec3(-36.5f, 2.0f, 98.5f),  // Point 1: Walk forward
    // ... 14 more points
};

enum P1State { P1_NONE, P1_LOOK, P1_PAUSE, P1_LOOK_BACK, P1_DONE
   };
P1State currentP1State = P1_NONE;
glm::vec3 savedDirection;
float timer = 0.0f;
```

Listing 9: Presentation Path Structure

Ghost Animation Data:

```
glm::vec3 ghostPos = glm::vec3(-36.5f, 0.5f, 83.5f);
glm::vec3 ghostTarget1 = glm::vec3(-59.5f, 0.5f, 83.5f);
glm::vec3 ghostTarget2 = glm::vec3(-59.5f, 0.5f, 88.5f);
glm::vec3 ghostTarget3 = glm::vec3(-42.5f, 6.5f, 62.0f);
bool ghostVisible = true;
bool ghostMovingToTarget1 = false;
```

```
float ghostSpeed = 0.07f;
float ghostRotationAngle = -90.0f;
```

Listing 10: Ghost Movement System

4.4 Class Hierarchy

The application follows an object-oriented design with clear separation of concerns:

4.4.1 Main Classes

Camera Class

- Maintain camera position, orientation (front/right/up vectors)
- Generate view matrix for rendering
- Process movement commands (WASD)
- Handle rotation via pitch/yaw angles

```
namespace gps {

    enum MOVE_DIRECTION {MOVE_FORWARD, MOVE_BACKWARD, MOVE_RIGHT,
        MOVE_LEFT};

    class Camera {

    public:
        Camera(glm::vec3 cameraPosition, glm::vec3 cameraTarget,
           glm::vec3 cameraUp);
        glm::mat4 getViewMatrix();
        void move(MOVE_DIRECTION direction, float speed);
        void rotate(float pitch, float yaw);

    public:
        glm::vec3 cameraPosition;
        glm::vec3 cameraTarget;
        glm::vec3 cameraFrontDirection;
        glm::vec3 cameraRightDirection;
        glm::vec3 cameraUpDirection;
    };
}
```

Listing 11: Camera Class Header

Model3D Class

- Load 3D models from OBJ files using *tiny_obj_loader.h* library
- Parse material definitions (MTL files) and load textures

- Store meshes and associated textures
- Provide draw interface for rendering

```
namespace gps {

    class Model3D {

    public:
        ~Model3D();
        void LoadModel(std::string fileName);
        void LoadModel(std::string fileName, std::string basePath
            );
        void Draw(gps::Shader shaderProgram);
        std::vector<gps::Mesh> meshes;

    private:
        std::vector<gps::Texture> loadedTextures;
        void ReadOBJ(std::string fileName, std::string basePath);
        gps::Texture LoadTexture(std::string path, std::string
            type);
        GLuint ReadTextureFromFile(const char* file_name);
    };
}
```

Listing 12: Model3D Class Header

Mesh Class

- Store vertex data (positions, normals, UVs) and indices
- Manage OpenGL buffer objects (VAO, VBO, EBO)
- Handle texture binding and material properties
- Execute draw calls

```
namespace gps {

    struct Vertex {
        glm::vec3 Position;
        glm::vec3 Normal;
        glm::vec2 TexCoords;
    };
    struct Texture {
        GLuint id;
        std::string type;
        std::string path;
    };
    struct Material {
        glm::vec3 ambient;
        glm::vec3 diffuse;
```

```
        glm::vec3 specular;
    };
    struct Buffers {
        GLuint VAO;
        GLuint VBO;
        GLuint EBO;
    };
    class Mesh {

    public:
        std::vector<Vertex> vertices;
        std::vector<GLuint> indices;
        std::vector<Texture> textures;

        glm::vec3 diffuseColor;
        Mesh(std::vector<Vertex> vertices, std::vector<GLuint>
           indices, std::vector<Texture> textures, glm::vec3
           diffuseColor);

        Buffers getBuffers();
        void Draw(gps::Shader shader);

    private:
        // Render data
        Buffers buffers;
        void setupMesh();
    };
```

Listing 13: Mesh Class Header

5 Graphical User Interface Presentation / User Manual

The application features a first-person 3D interface with minimal on-screen elements, allowing full immersion in the dungeon environment. All interaction occurs through keyboard and mouse controls, with console feedback for key actions.

5.1 Interface Overview

The interface consists of a fullscreen 3D viewport displaying the dungeon scene from the player's perspective. There are no visible GUI elements such as menus, health bars, or HUD overlays. The screen displays the rendered 3D environment with dynamic lighting and shadows.

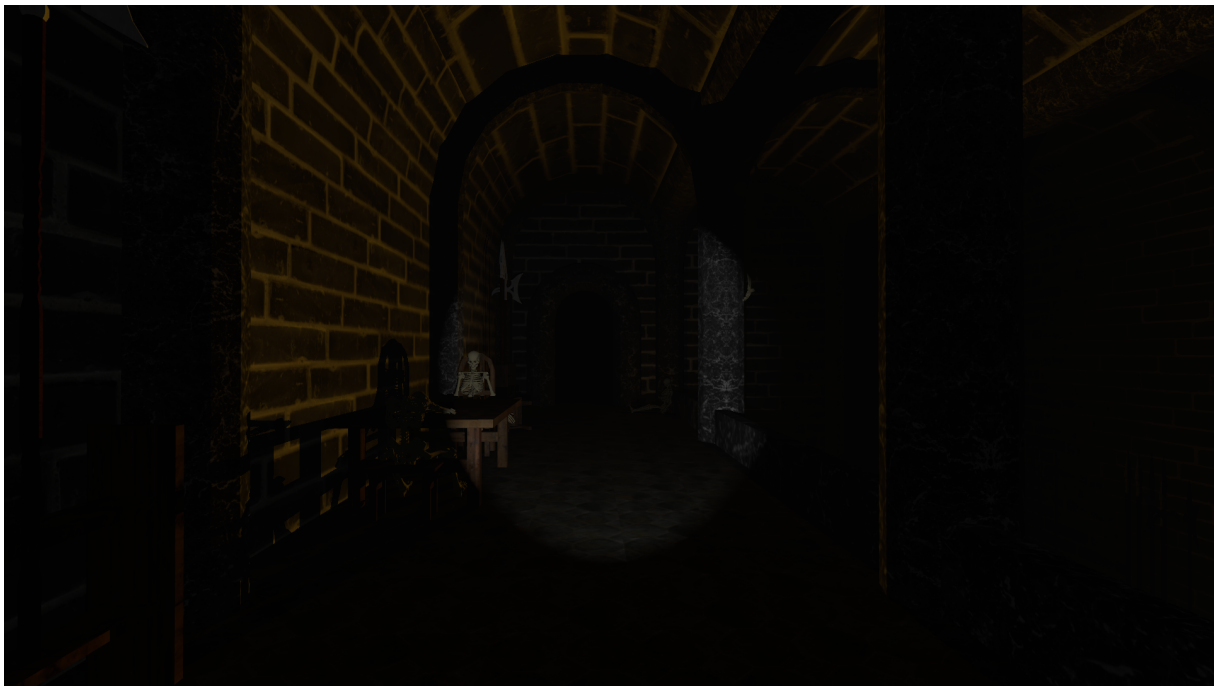

Figure 1: First Person View With Flashlight

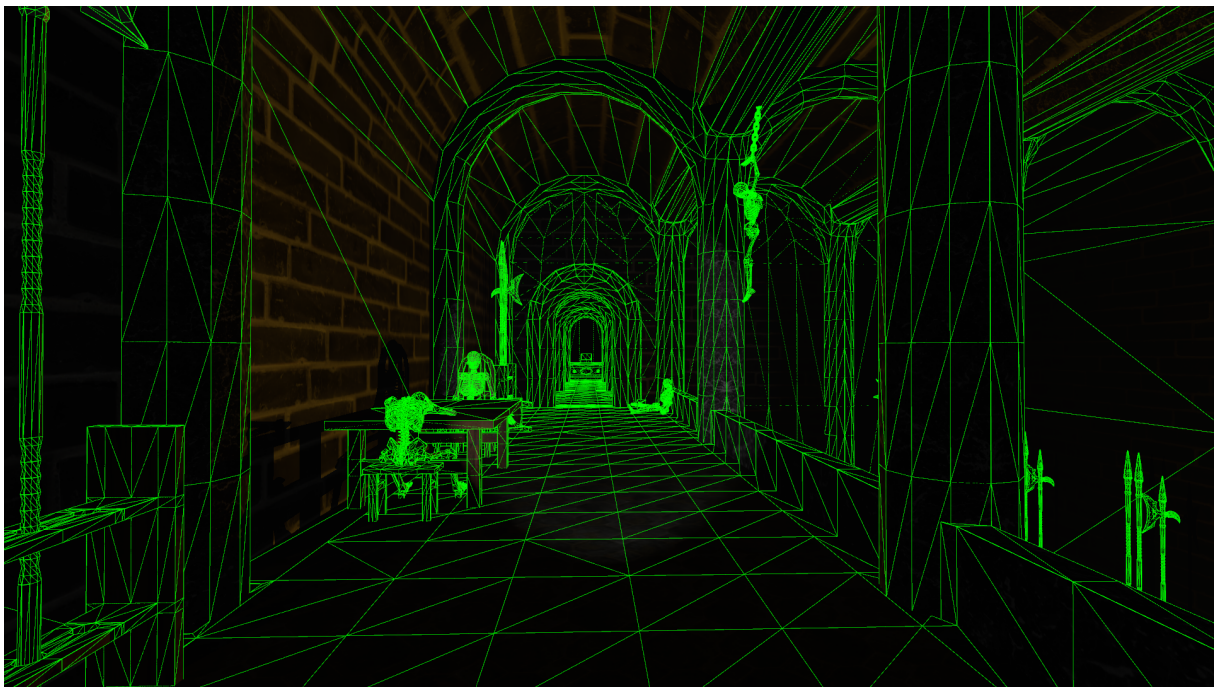

Figure 2: First Person View With Wireframe

5.2 User Manual

5.2.1 Getting Started

1. Launch the application - the window opens in borderless fullscreen mode
2. The camera starts at position (-36.5, 2.0, 114.5) in the starting corridor

3. Mouse cursor is captured and hidden for first-person control
4. Press **B** to start the automated presentation, or use WASD to explore freely

5.2.2 Basic Operations

- **Free Exploration**: Use WASD keys and mouse to navigate the dungeon manually. Collision detection prevents walking through walls.
- **Presentation Mode**: Press B to activate the automated camera path with cinematic sequences. The camera follows a predefined route showcasing lighting and architectural features.
- **Flashlight Toggle**: Press F to turn the player's flashlight on/off. Console displays "Flashlight: ON" or "Flashlight: OFF".
- **Debug Visualization**: Press C to toggle the green wireframe collision mesh overlay. Useful for understanding the spatial grid structure.

5.2.3 Keyboard Controls

Key	Action
W	Move forward
A	Strafe left
S	Move backward
D	Strafe right
F	Toggle flashlight on/off
C	Toggle collision wireframe on/off
B	Start/restart presentation mode
ESC	Exit application

Table 1: Keyboard controls

5.2.4 Mouse Controls

- **Look Around**: Move the mouse to rotate the camera view
- **Horizontal Movement**: Controls yaw (left/right rotation)
- **Vertical Movement**: Controls pitch (up/down rotation, clamped to ±89°)
- **Sensitivity**: 0.1 degrees per pixel - adjustable in code

6 Conclusions and Further Developments

6.1 Conclusions

The project successfully demonstrates advanced real-time rendering techniques in OpenGL, achieving photorealistic visuals through dual shadow mapping, efficient collision detection,

and atmospheric lighting. The spatial hash grid provides O(1) collision queries while maintaining a stable performance. The automated presentation mode effectively showcases all implemented features through a cinematic camera path with scripted events.

Key achievements include implementing two distinct shadow mapping systems (2D perspective and omnidirectional cubemap), a robust state-machine-driven camera animation system, and an efficient spatial data structure for collision detection. The modular class architecture allows easy extension and modification of scene elements.

6.2 Further Developments

Possible future enhancements include:

- **Dynamic Torch Animation**: Implement flickering light intensity and moving shadows to simulate realistic torch behavior
- **Additional Ghost Encounters**: Expand the presentation path with more scripted sequences and multiple ghost characters
- **Particle Systems**: Add dust particles in light shafts, torch embers, and ambient fog particles for enhanced atmosphere
- **Normal Mapping**: Implement normal mapping for stone walls to add surface detail without increasing polygon count
- **Audio Integration**: Add spatial audio for footsteps or ambient dungeon sounds
- **Performance Optimizations**: Implement frustum culling and occlusion culling to handle larger, more complex dungeon environments

7 References

References

[1] Online Resource, `https://opengameart.org/content/skeleton-with-rig`, The main skeleton model used throughout the scene.

[2] Online Resource, `https://ambientcg.com/`, The materials used for texturing the different models and objects.